

Project Database Proposal

Eduardo Garcia

Department of Computer Science: Saint Xavier University

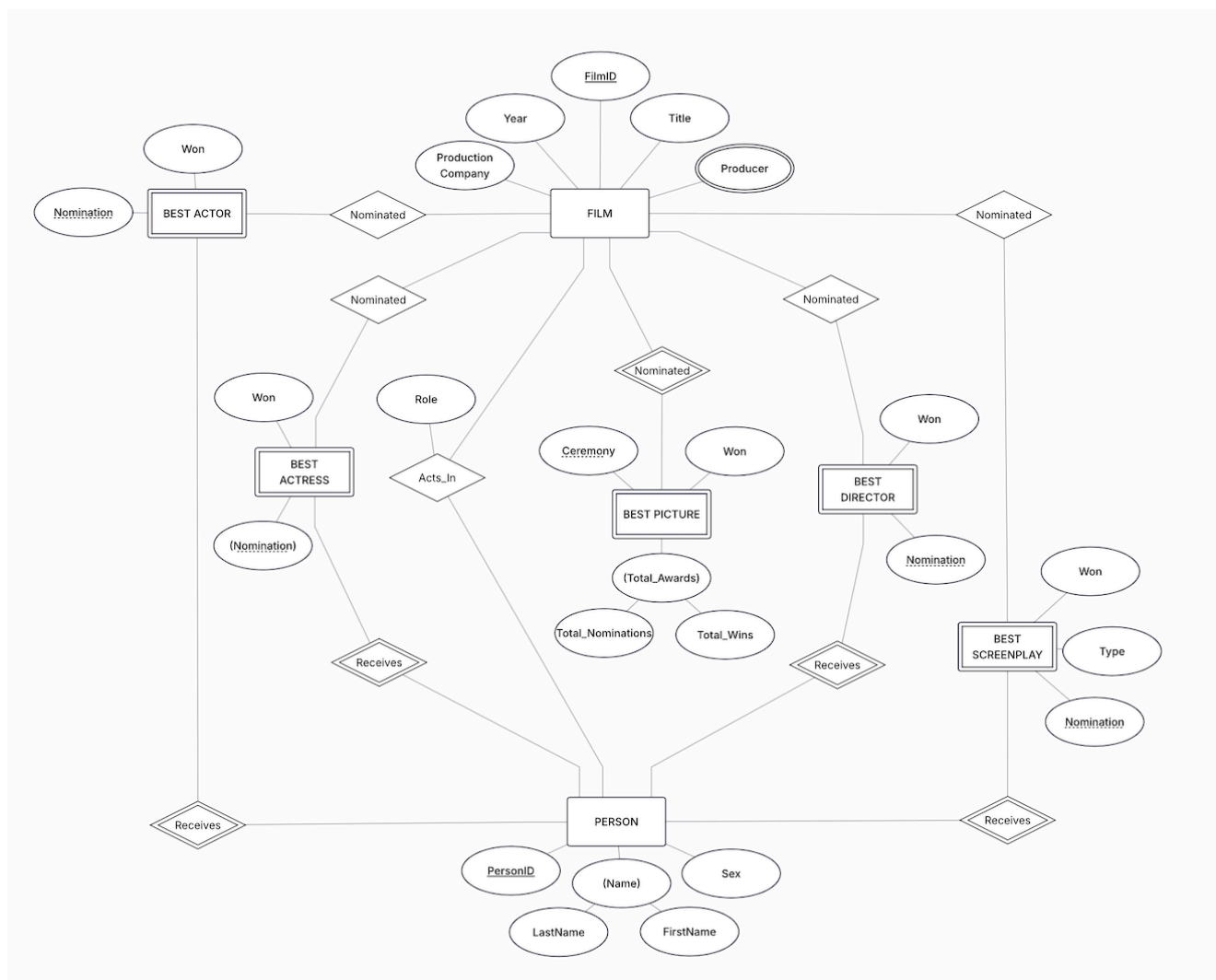
CMPSC-321:Relational Database Theory and Design

Dr. James Vanderhyde

December 2, 2025

Project Proposal: The domain for which my database is the Big 5 Academy Award winners and nominees. Big5 is a structural database that traces all films nominated at the Academy Awards for their 5 above the line categories including Best Picture, Best Director, Best Actor, Best Actress, and Best Screenplay (Original or Adapted). This database includes 7 entities which are Film, Person, Best Picture, Best Director, Best Actor, Best Actress, and Best Screenplay. The ER diagram would describe the relationship these nominations have with one another and accounting for a film and its actors, director(s), and writer(s) that are nominated in their respective category for each year this instance occurred.

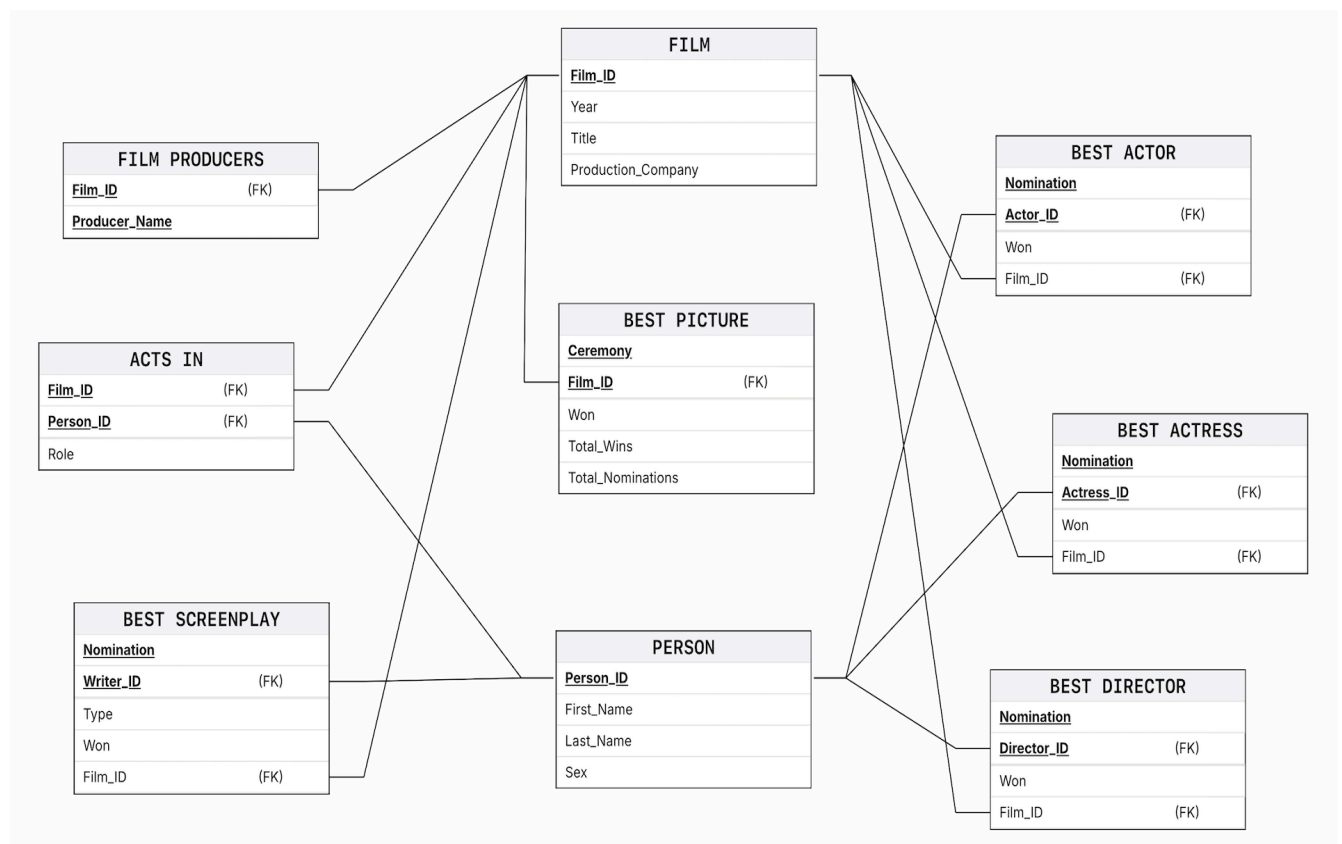
ER DIAGRAM 3NF



Applied Chen notation to design ER diagrams and relational schemas that were later converted into MySQL tables for the database. Film and Person represent strong entities while the other 5 entities that represent the big 5 categories of the Academy Awards function as weak entities with

[nomination] as their partial identifier that eventually pairs with the primary key of Person to fully be a unique identifier. They exist as weak entities because they depend on the Person and Film existing. For example, the Best Director entity had no unique identity aside from the partial identifier that is [nomination] for a certain instance (number) a person received a nod. The nomination entities are modeled as weak entities because they have no independent identity. A director nomination cannot exist without a film and a person, and its identity derived from those identifying entities together. This is true for every weak entity present in the diagram and database schema.

RELATIONAL SCHEMA 3NF



The relational schema showcases all tables and their relationships to one another. In addition, showcasing each table's primary key in bold underline and its foreign key (FK) meaning what table it's referencing.

TABLES WITH ENTITIES AND ATTRIBUTES

FILM

<u>FILM ID</u>	<u>YEAR</u>	<u>TITLE</u>	<u>PRODUCTION COMPANY</u>

PERSON

<u>PERSON ID</u>	<u>FIRST NAME</u>	<u>LAST NAME</u>	<u>SEX</u>

PRODUCERS

<u>FILM ID</u>	<u>PRODCUER NAME</u>

ACTS IN

<u>PERSON ID (FK)</u>	<u>FILM ID (FK)</u>	<u>ROLE</u>

BEST PICTURE

<u>CEREMONY</u>	<u>FILM ID (FK)</u>	<u>TOTAL_WINS</u>	<u>TOTAL_NOMINATIONS</u>	<u>WON</u>

BEST ACTOR

<u>NOMINATION</u>	<u>ACTOR ID (FK)</u>	<u>FILM ID (FK)</u>	<u>WON</u>

BEST ACTRESS

<u>NOMINATION</u>	<u>ACTRESS ID (FK)</u>	<u>FILM ID (FK)</u>	<u>WON</u>

BEST DIRECTOR

<u>NOMINATION</u>	<u>DIRECTOR ID (FK)</u>	<u>FILM ID (FK)</u>	<u>WON</u>

BEST SCREENPLAY

<u>NOMINATION</u>	<u>WRITER ID (FK)</u>	<u>FILM ID (FK)</u>	<u>TYPE</u>	<u>WON</u>

CREATING SCHEMAS ON MYSQL

```
CREATE DATABASE IF NOT EXISTS big5db;
```

```
USE big5db;
```

```
CREATE TABLE FILM (  
    Film_ID CHAR(4) NOT NULL,  
    Year INT NOT NULL,  
    Title VARCHAR(100) NOT NULL,  
    Production_Company VARCHAR(50) NOT NULL,  
    PRIMARY KEY (Film_ID)  
);
```

```
CREATE TABLE FILM_PRODUCERS (  
    Producer_Name VARCHAR(100) NOT NULL,  
    Film_ID CHAR(4) NOT NULL,  
    PRIMARY KEY (Film_ID, Producer_Name),  
    FOREIGN KEY (Film_ID) REFERENCES FILM(Film_ID)  
);
```

```
CREATE TABLE PERSON (  
    Person_ID CHAR(5) NOT NULL,  
    First_Name VARCHAR(50) NOT NULL,  
    Last_Name VARCHAR(50) NOT NULL,  
    Sex VARCHAR(10) NOT NULL,  
    PRIMARY KEY (Person_ID),  
    CONSTRAINT CHECK_Sex  
    CHECK (Sex in ("Male","Female","Non-Binary"))  
);
```

```
CREATE TABLE BEST_PICTURE (  
    Ceremony CHAR(5) NOT NULL,  
    Film_ID CHAR(4) NOT NULL,  
    Total_Wins INT NOT NULL,  
    Total_Nominations INT NOT NULL,  
    Won CHAR(3) NOT NULL,  
    PRIMARY KEY (Ceremony, Film_ID),  
    FOREIGN KEY (Film_ID) REFERENCES FILM(Film_ID),  
    CONSTRAINT CHECK_Won CHECK (Won in ("Yes","No"))  
);
```

```
CREATE TABLE BEST_ACTOR (  
  Nomination INT NOT NULL,  
  Actor_ID CHAR(5) NOT NULL,  
  Film_ID CHAR(4) NOT NULL,  
  Won CHAR(3) NOT NULL,  
  PRIMARY KEY (Nomination, Actor_ID),  
  FOREIGN KEY (Actor_ID) REFERENCES PERSON(Person_ID),  
  FOREIGN KEY (Film_ID) REFERENCES FILM(Film_ID),  
  CONSTRAINT CHECK_Won2  
  CHECK (Won in ("Yes","No"))  
);
```

```
CREATE TABLE BEST_ACTRESS (  
  Nomination INT NOT NULL,  
  Actress_ID CHAR(5) NOT NULL,  
  Film_ID CHAR(4) NOT NULL,  
  Won CHAR(3) NOT NULL,  
  PRIMARY KEY (Nomination, Actress_ID),  
  FOREIGN KEY (Actress_ID) REFERENCES PERSON(Person_ID),  
  FOREIGN KEY (Film_ID) REFERENCES FILM(Film_ID),  
  CONSTRAINT CHECK_Won3  
  CHECK (Won in ("Yes","No"))  
);
```

```
CREATE TABLE BEST_DIRECTOR (  
  Nomination INT NOT NULL,  
  Director_ID CHAR(5) NOT NULL,  
  Film_ID CHAR(4) NOT NULL,  
  Won CHAR(3) NOT NULL,  
  PRIMARY KEY (Nomination, Director_ID),  
  FOREIGN KEY (Director_ID) REFERENCES PERSON(Person_ID),  
  FOREIGN KEY (Film_ID) REFERENCES FILM(Film_ID),  
  CONSTRAINT CHECK_Won4  
  CHECK (Won in ("Yes","No"))  
);
```

```

CREATE TABLE BEST_SCREENPLAY (
  Nomination INT NOT NULL,
  Writer_ID CHAR(5) NOT NULL,
  Film_ID CHAR(4) NOT NULL,
  Type CHAR(8) NOT NULL,
  Won CHAR(3) NOT NULL,
  PRIMARY KEY (Nomination, Writer_ID),
  FOREIGN KEY (Writer_ID) REFERENCES PERSON(Person_ID),
  FOREIGN KEY (Film_ID) REFERENCES FILM(Film_ID),
  CONSTRAINT CHECK_Won5
  CHECK (Won in ("Yes","No")),
  CONSTRAINT CHECK_Type
  CHECK (Type in ("Original","Adapted"))
);

```

```

CREATE TABLE ACTS_IN (
  Person_ID CHAR(5) NOT NULL,
  Film_ID CHAR(4) NOT NULL,
  Role VARCHAR(100) NOT NULL,
  PRIMARY KEY (Film_ID, Person_ID),
  FOREIGN KEY (Film_ID) REFERENCES FILM(Film_ID),
  FOREIGN KEY (Person_ID) REFERENCES PERSON(Person_ID)
);

```

SQL QUERIES

/* Right Join Query showcases a film contents with its best_picture win and total awards/nominations */

```

SELECT F.Film_ID, BP.Ceremony, F.Year, F.Title, BP.Total_Wins, BP.Total_Nominations,
BP.Won
FROM FILM AS F
RIGHT JOIN BEST_PICTURE as BP
ON F.FILM_ID = BP.FILM_ID
ORDER BY YEAR ASC;

```

/* Join Querie showcases best_actor nominations for a given Actor joining person, film, and best_actor tables */

```

SELECT F.Film_ID, F.Year, F.Title, BA.Nomination, BA.Actor_ID, P.First_Name,
P.Last_Name, BA.Won
FROM FILM AS F, PERSON AS P, BEST_ACTOR AS BA

```

```
WHERE F.FILM_ID = BA.FILM_ID AND P.PERSON_ID = BA.ACTOR_ID
ORDER BY F.YEAR ASC;
```

/* Join Query showcases best_actress nominations for a given Actress joining person, film, and best_actress tables */

```
SELECT F.Film_ID, F.Year, F.Title, BA.Nomination, BA.Actress_ID, P.First_Name,
P.Last_Name, BA.Won
FROM FILM AS F, PERSON AS P, BEST_ACTRESS AS BA
WHERE F.FILM_ID = BA.FILM_ID AND P.PERSON_ID = BA.ACTRESS_ID
ORDER BY YEAR ASC;
```

/* Join Query showcases best_director nominations for a given director joining person, film, and best_director tables */

```
SELECT F.Film_ID, F.Year, F.Title, BD.Nomination, BD.Director_ID, P.First_Name,
P.Last_Name, BD.Won
FROM FILM AS F, PERSON AS P, BEST_DIRECTOR AS BD
WHERE F.FILM_ID = BD.FILM_ID AND P.PERSON_ID = BD.DIRECTOR_ID
ORDER BY YEAR ASC;
```

/* Join Query showcases best_screenplay nominations for a given writer joining person, film, and best_screenplay tables */

```
SELECT F.Film_ID, F.Year, F.Title, BS.Nomination, BS.Writer_ID, P.First_Name,
P.Last_Name, BS.Won
FROM FILM AS F, PERSON AS P, BEST_SCREENPLAY AS BS
WHERE F.FILM_ID = BS.FILM_ID AND P.PERSON_ID = BS.Writer_ID
ORDER BY F.film_id ASC;
```

/* Query showcases total number of times a film producer has produced a film in the db*/

```
SELECT producer_name, count(producer_name) as total
FROM FILM_PRODUCERS
group by producer_name
order by total desc;
```

/* Query showcases total number of times a production company has produced a film in the db*/

```
SELECT PRODUCTION_COMPANY, count(PRODUCTION_COMPANY) as total
FROM FILM
group by PRODUCTION_COMPANY
order by total desc;
```